

Dehradun Waste Watch: Technical Documentation & Implementation Workflow

1. Project Overview

Dehradun Waste Watch is an AI-powered urban waste management system designed to streamline the reporting, tracking, and resolution of waste-related issues in Dehradun. It leverages state-of-the-art computer vision models and a robust web architecture to bridge the gap between citizens and municipal authorities.

2. System Workflow

The system follows a structured lifecycle from the moment a waste issue is spotted to its final resolution.

1. **Authentication:** Users (Citizens, Administrators, or Authorities) register and log in to the platform.
2. **Reporting:** A citizen uploads an image of waste. The system automatically captures the GPS coordinates (or allows manual input) and the locality.
3. **AI Analysis:**
 - The image is processed using **SAHI (Slicing Aided Hyper Inference)** to detect small objects like cigarette butts or plastic caps that are often missed by standard detectors.
 - A custom **YOLOv8 model injected with MSSA (Multi-Scale Spatial Attention)** performs the detection.
 - **COCO Gating** is applied to filter out false positives by checking if the detected waste overlaps with known non-waste objects (like cars or animals).
4. **Automatic Assignment:** Based on the locality (e.g., Rajpur Road, Clock Tower), the report is automatically assigned to the relevant Nagar Nigam ward or Cantonment Board authority.
5. **Notification:** Real-time updates are sent via WebSockets to active dashboards, and email notifications are triggered for both the reporter and the authority.
6. **Resolution:** The assigned authority views the report, updates the status (Reported -> Taken Up -> Being Solved -> Solved), and uploads a “resolution photo” once the site is cleaned.
7. **Verification:** The citizen can verify the resolution and provide feedback or mark it as disputed if the cleaning was insufficient.

3. Core Components

- **FastAPI Backend (app.py):** Handles API requests, model inference, and real-time communication.

- **SQLAlchemy Models (`models.py`):** Defines the relational database schema for users and waste reports.
- **MSSA Attention Layers (`mssa_layers.py`):** Custom PyTorch implementation of Multi-Scale Spatial Attention injected into the YOLOv8 neck to enhance feature extraction.

4. Key Functions and Logic

AI Inference & Enhancement

- **`inject_mssa(model)`:** Dynamically modifies the YOLOv8 architecture at runtime, inserting MSSA blocks after C2f layers in the neck to improve detection of varied-size waste.
- **`get_sahi_predictions(image)`:** Implements Slicing Aided Hyper Inference, which slices high-resolution images into smaller overlapping patches, runs inference on each, and merges the results. This is critical for detecting “micro-waste”.
- **`apply_coco_gating(predictions, image)`:** Uses a standard YOLOv8m model trained on the COCO dataset to identify common objects. If a “waste” detection significantly overlaps with a “non-waste” object (like a person or vehicle), it is suppressed to improve precision.

Report Management

- **`create_report()`:** Validates the upload, runs the AI pipeline, saves the image, creates a database entry, and triggers background tasks for notifications.
- **`update_report_status()` / `authority_update_report()`:** Manages the state transitions of a report and handles the storage of “after” images.
- **`_serialize_report(r)`:** A helper function that formats database objects into JSON-friendly dictionaries, including authority contact details mapped from locality data.

5. Source Code Implementation: `app.py`

```
$(cat app.py)
```